Reproducibility Challenge: Overcoming Catastrophic Forgetting in Neural Networks

Jan JURASZEK, Dimitri PELGRIN, Yann SALVIGNOL, Veronika WANNACK
EPFL, EE-411: Fundamentals of inference and learning

***Abstract*—When neural networks train sequentially on multiple tasks, learning new tasks tends to erase knowledge of previous ones. This phenomenon, called catastrophic forgetting, remains an important challenge in continual learning. In this work, we reproduce the Elastic Weight Consolidation (EWC) algorithm proposed by Kirkpatrick et al. 2017. EWC selectively constrains important parameters using a Fisher Information-based penalty. We conduct experiments on two datasets: Permuted MNIST and Rotated MNIST. Our experiments with permuted MNIST show that EWC is effective at reducing catastrophic forgetting in comparison to SGD or L2 regularization, as the accuracy on the first task remains above 80% even after training on 9 subsequent tasks. In comparison, SGD and L2 degrade to 60%. However, on Rotated MNIST, EWC drops to below 80% and all three methods show a similar degradation over time. This observation reveals a limitation of EWC: the method performs better on independent tasks than on correlated ones, and struggles when tasks share underlying features. Our findings confirm the original claims of the paper while showing where this method may fail.**

I. Introduction

A key property of artificial general intelligence is the ability to continually learn: an agent should be capable of learning new tasks without forgetting previous ones. However, despite the apparent simplicity of this statement, the vast majority of artificial neural networks do not adhere to it. McCloskey and Cohen McCloskey and Cohen 1989 labeled this problematic phenomenon "**catastrophic forgetting**". Simply put, it refers to the tendency of a neural network to overwrite parameters learned during previous training iterations while learning how to perform a new task.

Previous attempts at solving this issue yielded moderate results and raised concerns about memory usage which grows as the number of sequential task increases.

The authors of the selected paper aim to mimic the neurological mechanisms of dendritic spines, which are parts of a neuron that are thought to be responsible for learning in mammals. In particular, spine formation increases when animals learn new tasks. Interestingly, if the spines are erased, the corresponding skills are forgotten. As such, the plasticity of neurons determines whether learned skills can withstand subsequent training. Therefore, the suggested approach is to vary the "elasticity" of each neuron depending on its importance for learning a specific task.

On the one hand, rigid parameters do not possess sufficient flexibility to adapt to new tasks, and on the other hand, excessively elastic parameters would fall into the *catastrophic forgetting* trap. In order to tackle this compromise, the algorithm called **Elastic Weight Consolidation (EWC)**, anchors parameters while allowing "elastic" spring-like adaptations.

II. Methodology

A. Data Preparation and Task Construction

The standard MNIST dataset was transformed into a sequence of distinct tasks. The following methods create tasks of equal complexity that reside in different "input domains", forcing the network to learn new mappings without changing the output space.

a) Permuted MNIST Generation: Task 1 is the original MNIST dataset. For subsequent tasks (Task 2 to Task N), a fixed random permutation is applied to the input pixels. The 28×28 images are first flattened to a 784 vector, then the indices are randomly shuffled. Implementation is provided in `get_permuted_mnist_tasks` in the Jupyter notebook.

b) Rotated MNIST Generation: Task 1 is the original MNIST dataset. For subsequent tasks (Task 2 to Task N, indexed $i = 2 \cdots N$), a counter-clockwise rotation of $10 \times (i - 1)$ degrees is applied to each image, and then flattened to a 784 vector. Implementation is provided in `get_rotated_mnist_tasks` in the Jupyter notebook.

B. Model Architecture

The implemented neural network is a multi-layer perceptron (MLP) of 784-unit input, 10-unit linear output and two 400-unit ReLU hidden layers. This architecture was chosen to balance network capacity and computational efficiency, as studied during the semester. The implementation of the MLP is provided in the `class MLP`.

C. Elastic Weight Consolidation (EWC) Implementation

a) Approximation of the Posterior: The implementation approximates the intractable posterior distribution of the network parameters $p(\theta|\mathcal{D})$ as a Gaussian distribution centered at the optimal parameters found in the previous tasks.

b) Fisher Information Matrix (FIM) $\mathcal{F}$: $\mathcal{F}$ is calculated to quantify the importance of each parameter to the previously learned task. Parameters with high Fisher values are constrained more strictly, while those with low values remain plastic. Computationally, it is only feasible to compute the diagonal of the matrix $\mathcal{F}$, rather than the full matrix. $\mathcal{F}$ is computed at the end of each task by averaging the squared gradients of the loss (negative log-likelihood) over a subset of the training data; 500 samples in our case.

c) Augmented Loss Function: Training on a new task involves minimizing a composite objective function that adds a quadratic penalty to the standard Cross-Entropy loss. Specifically, the loss function used is:

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} \mathcal{F}_i (\theta_i - \theta^*_{A,i})^2$$

Here, $\mathcal{L}_B(\theta)$ is the loss for the current task, λ is a hyperparameter regulating the trade-off between plasticity (learning new tasks) and stability (remembering old tasks), $\mathcal{F}_i$ is the FIM value for parameter i and $\theta^*_{A,i}$ is the optimal parameter value from the previous task.

d) Sequential Task Registration: Upon completing a task, the current parameters of the model and their computed Fisher values are stored. For subsequent tasks, the penalty term iterates over all previously stored tasks, summing their respective quadratic constraints to prevent catastrophic forgetting across the entire sequence. Contrary to the original paper, our implementation normalizes the total penalty by the number of tasks to maintain constant regularization strength. Without this normalization, the cumulative penalty eventually dominates the task loss, preventing new learning and causing severe performance degradation around task 5-7.

D. Baseline Comparison

The effectiveness of EWC is validated by comparison with two common approaches:

a) Sequential SGD: Training the model sequentially on tasks using standard Stochastic Gradient Descent without any regularization, a method which suffers from the "catastrophic forgetting" phenomenon.

b) L2-Regularization & Weight Decay: Training with a quadratic penalty that constrains parameters to remain close to their values from the previous task. This is equivalent to EWC with a uniform Fisher matrix (F=I), assuming all parameters are equally important. Unlike EWC's selective weighting, this method applies the same regularization strength to all parameters regardless of their importance to previous tasks. Naive L2 regularization toward zero did not reproduce the original paper's results, so we implemented L2 toward previous parameters instead.

E. Training and Evaluation Protocol

At the end of every epoch, the model is evaluated on the test sets of **all tasks** (past and present), allowing us to track catastrophic forgetting as training progresses. The optimizer is reinitialized at the start of each task. Experiments used the parameters in Table I.

Training		**Regularization**	
Optimizer	SGD (reset/task)	λ_{EWC}	200
Learning rate	0.1	λ_{L2}	0.001
Batch size	128	Fisher samples	500
Epochs/task	5	Random seed	42

TABLE I
EXPERIMENTAL PARAMETERS.

III. RESULTS

A. Results on Permuted MNIST tasks

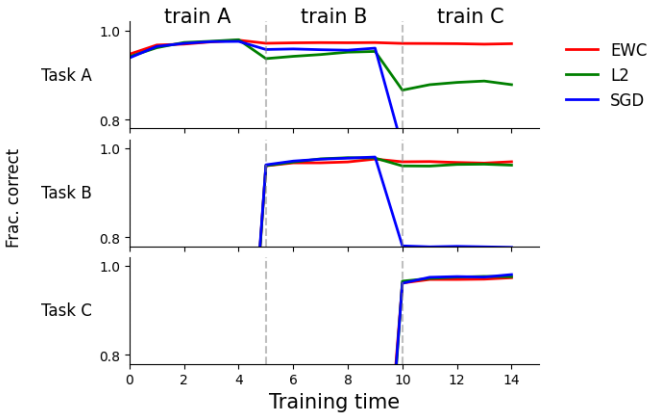


Fig. 1. Reproduction of Figure 2A on the permuted dataset

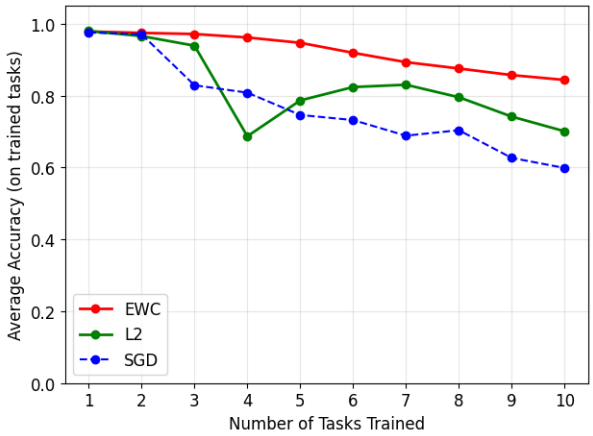


Fig. 2. Reproduction of Figure 2B on the permuted dataset

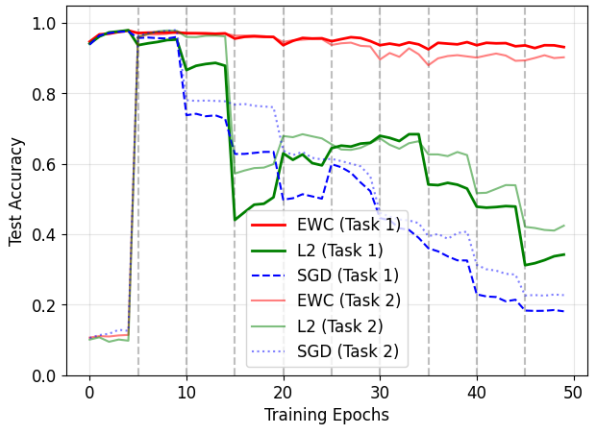


Fig. 3. Accuracy progression of tasks 1 and 2 on the permuted dataset

B. Results on Rotated MNIST tasks

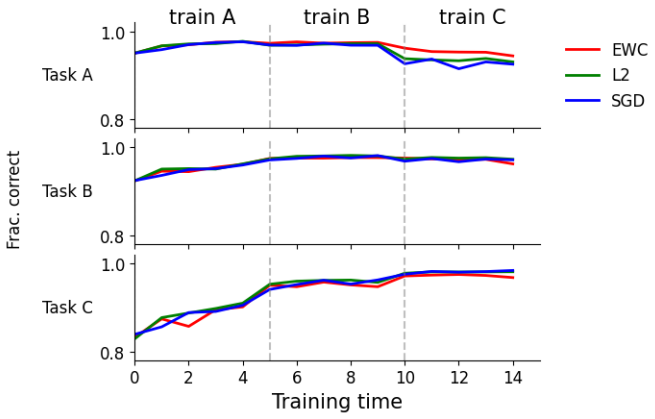


Fig. 4. Reproduction of Figure 2A on the rotated dataset

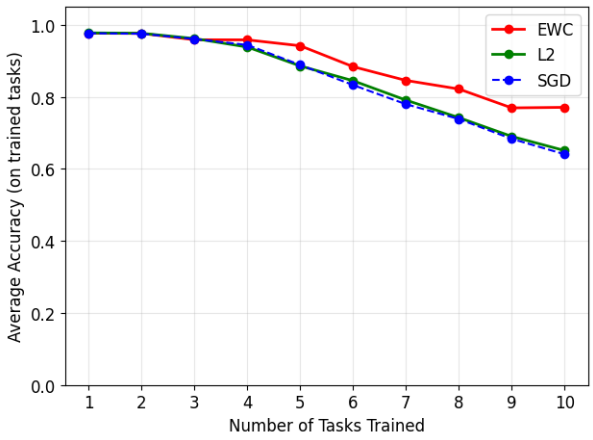


Fig. 5. Reproduction of Figure 2B on the rotated dataset

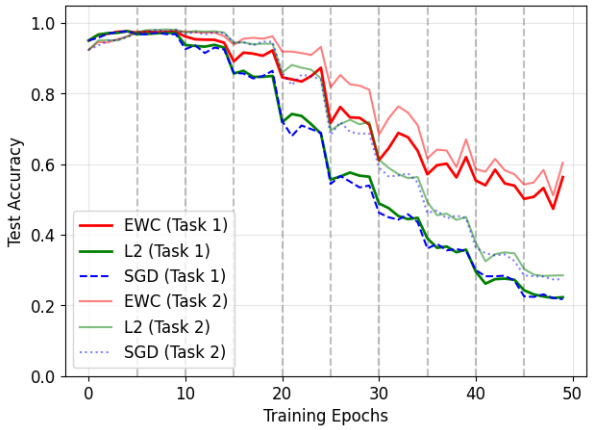


Fig. 6. Accuracy progression of tasks 1 and 2 on the rotated dataset

IV. Discussion

A. Description of Results

1) Experiment 1: Permuted MNIST: The results for the Permuted MNIST benchmark are presented in Figures 1, 3 and 2. Plot 3 tracks the test accuracy on the original MNIST dataset (Task A) while the model is trained sequentially on later tasks.

a) Evidence of Catastrophic Forgetting: The trajectory of the standard SGD baseline provides a clear visualization of catastrophic forgetting. As soon as training on Task B begins, the accuracy on Task A drops sharply and continues to fall as training progresses. This confirms that without constraints, the optimization procedure completely overwrites the weights required for Task A in order to minimize the loss for Task B.

b) Performance of EWC: In contrast, the EWC implementation demonstrates strong stability and maintains high accuracy (>80%) throughout the training sequence. Contrary to the original paper, this EWC implementation degrades in accuracy as more tasks are added, which could be expected as the parameter space becomes increasingly constrained. However, the gap between the EWC curve and SGD/L2 shows the effectiveness of EWC in protecting specific parameters relevant to earlier tasks.

c) Comparison with L2-Regularization: The L2 baseline shows intermediate but unstable performance and finishes better than SGD but worse than EWC.

d) Hyperparameters: EWC proved to be very sensitive to hyperparameter choice. Performance varied drastically with λ_{EWC} and learning rate, and certain combinations showed unstability in EWC. Despite extensive hyperparameter tuning, testing different normalizations, or longer training, we were unable to exactly reproduce the original paper's results, since many implementation details were not specified.

2) Experiment 2: Rotated MNIST: Figures 4, 6 and 5 illustrates the results for the Rotated MNIST tasks. This scenario tests the network's ability to handle geometric transformations rather than uncorrelated permutations.

Unlike Permuted MNIST, EWC shows a smaller improvement over baselines on Rotated MNIST, while the baseline shows better stability particularly with L2 regularization. This indicates a limitation of EWC when dealing with tasks sharing semantic features (eg. edges, curves), which require conflicting parameter values, causing the the Fisher-weighted constraints to prevent necessary parameter updates.

Evidence of feature sharing is visible in the early training phases: untrained rotated tasks achieve high accuracy ($> 80\%$) because they share structural features with already-trained tasks. For instance, a network trained only on $0°$ rotation already achieves $> 90\%$ on the $20°$ task. This contrasts with Permuted MNIST, where untrained tasks remain at chance level ($\approx 10\%$).

B. Interpretation of Results

1) Qualitative analysis: The primary objective of the project was to determine if the "catastrophic forgetting" phenomenon could be mitigated using EWC. The results on the

Permuted (and partially the Rotated) MNIST confirm that standard SGD creates a greedy optimization path. When the network shifts to a new task, SGD drastically alters weights to minimize the new loss, ignoring the "error regions" created for previous tasks. In contrast, EWC successfully retained accuracy on early tasks while learning new ones. In this sense, the core hypothesis of Kirkpatrick et al. 2017 is validated: neural networks are sufficiently over-parameterized to contain subspaces able to satisfy multiple tasks simultaneously. Hence, EWC acts as a tool to map the different intersecting solution subspaces. The success of EWC relies entirely on the geometry of the loss landscape, which is approximated by the FIM.

2) Comparison with Baselines (L2-Regularization): Comparing EWC with the L2-regularization baseline demonstrates the importance of selectivity. L2-regularization effectively assumes that all parameters are equally important, which leads to unstable performance: on Permuted MNIST, L2 drops sharply before partially recovering. On Rotated MNIST, however, L2 is smooth, implying that uniform regularization is enough if tasks share some features, and EWC provides a clear advantage only when tasks are independent.

3) Choice of Hyper-parameters: A critical factor in the conducted experiments was the hyperparameter λ. If λ is chosen too low, the consolidation is too weak and the network behaves like standard SGD, essentially overwriting old skills. If λ is chosen too high, the network becomes over-constrained: while it might be able to perfectly align with the first task, it lacks the flexibility to minimize the error of subsequent tasks. It may also happen that the accuracy plummets with a very high λ, the limit of which may be raised by using normalization as explained in $II.C.d$. The range of λ varied between 50 and 400 in our exploration, and a value of $\lambda = 200$ was empirically determined as a "sweet spot".

4) Possible Limitations:

a) Task boundaries: In the suggested implementation, there are explicit task signals. The FIM computation is manually triggered at the end of every epoch. In a realistic "continual learning" scenario, the agent would not necessarily know when a task switches.

b) Scalability: The FIM requires computing second-order derivatives (or at least approximating them by squared gradients) and storing a value for each parameter. As the network size grows, this memory cost could become prohibitive compared to simpler methods.

V. Conclusion

We have successfully reproduced the results of Kirkpatrick et al. 2017, Figure 2 A and Figure 2 B. Results on Permuted MNIST show that EWC efficiency reduces catastrophic forgetting, in contrast to SGD and L2 regularization. This supports the hypothesis of the paper, that over-parameterized networks have the capacity to solve multiple tasks simultaneously if the most important weights are protected.

However, testing on Rotated MNIST shows the limitations of EWC. The characteristic of Permuted MNIST is that the created images are random, they do not share the same subspace of parameters. However, the different tasks in Rotated MNIST have similar structures, such as similar edges and curves. In this scenario, EWC performs worse: the Fisher Information Matrix finds the most important parameters for recognizing specific features, but these same parameters need different values for differently-rotated inputs. These conflicts prevent the neural network from adapting to the new data, which reduces the overall test accuracy. Thus, EWC is limited by task independence: different tasks with similar features will perform worse than orthogonally independent tasks.

Further works might address different calculations of importance through correlation, such as full Fisher matrices with parameter correlations. From a practical standpoint, our reproduction shows the effectiveness of EWC in continual learning, but applying it in practice needs careful consideration of task relationships.

References

Kirkpatrick, James et al. (2017). "Overcoming catastrophic forgetting in neural networks". In: *Proceedings of the National Academy of Sciences* 114.13, pp. 3521–3526. DOI: 10.1073/pnas.1611835114.

McCloskey, Michael and Neal J. Cohen (1989). "Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem". In: *Psychology of Learning and Motivation*. Vol. 24. Academic Press, pp. 109–165. DOI: 10.1016/S0079-7421(08)60536-8. URL: https://www.sciencedirect.com/science/article/pii/S0079742108605368.

Appendix

Team Contributions

Jan Juraszek: Optimizing EWC/L2 implementations, figure generation and formatting, documentation of improvements.

Dimitri Pelgrin: Worked on the first iteration of the MLP, the Fisher matrix and the training loop. Wrote the introduction, the methodology and part of the discussion section.

Yann Salvignol: Rotated MNIST dataset implementation, hyperparameter tuning for result accuracy, validating early iterations of the EWC algorithm.

Veronika Wannack: Early iterations of EWC algorithm and literature review, preprocessing and task generation, hyperparameter testing, writing the discussion, code and report cleanup, experimental validation.

Source Code Notebook

The Jupyter notebook containing all experiments is submitted together with this report. To run the code:

1) Install dependencies: `torch`, `torchvision`, `matplotlib`, `numpy`
2) Set `MNIST_DATA_PATH` to your local directory (or set `download=True` if there is no local directory yet)
3) Run cells sequentially

Running all tasks on CPU takes approximatively 30 minutes. Running on Google Colab with GPUs is recommended.